



HİPOTEZ TESTLERİ ÖDEV 2

AHMET ENES KUZU

2230329059

ÖDEV 1

```
import numpy as np
import matplotlib.pyplot as plt
from numpy import random

np.random.seed(1)

kitle= random.normal(loc=50,scale=10,size=10000)
print(kitle)
print()
kitle_ort=np.mean(kitle)
print(kitle_ort)
kitle_var=np.var(kitle)
print(kitle_var)

[66.24345364 43.88243586 44.71828248 ... 39.85856184 49.37303774
 35.62130107]

50.09772656699105
99.75731615516466
```

KODUN AÇIKLAMASI: np.random.seed() her çalıştırmada farklı sonuçlar almamak için kullanılan kod. Genel olarak 42 kullanılır ama isteyen istediği sayıyı kullanabilmektedir. Rastgele ortalaması 50, standart sapması 10 olan 10000 büyüklüğünde (normal dağılıma uygun) dağılıma rastgele bir kitle çekilmiştir. Kitlenin ortalaması ve varyansı numpy kütüphanesi np.mean ve np.var kodlarıyla ortalama ve varyans bulunmuştur.

YORUM: Simülasyon sonucunda seçilen 10000 tane gözlemin ortalaması ve varyansı kitlenin ortalaması ve varyansına yakınsar. Örneklem büyüklüğü arttıkça elde edilen istatistiklerin parametreye yakınsaması beklenmektedir.

```
orneklem1=np.random.choice(kitle, size=10, replace=False)
orneklem_ort=np.mean(orneklem1)
print(orneklem_ort)
orneklem_var=np.var(orneklem1)
print(orneklem_var)
print()

orneklem2=np.random.choice(kitle, size=30, replace=False)
orneklem_ort=np.mean(orneklem2)
print(orneklem_ort)
orneklem_var=np.var(orneklem2)
print(orneklem_var)
print()

orneklem3=np.random.choice(kitle, size=100, replace=False)
orneklem_ort=np.mean(orneklem3)
print(orneklem_ort)
orneklem_var=np.var(orneklem3)
print(orneklem_var)
print()

orneklem4=np.random.choice(kitle, size=500, replace=False)
orneklem_ort=np.mean(orneklem4)
print(orneklem_ort)
orneklem_var=np.var(orneklem4)
print(orneklem_var)
print()

49.13459724564386
52.60782209982877

46.1953427833323
131.59611085404873

47.7222188867498
100.99339920029381

50.69480208819548
111.50305198749362
```

KODUN AÇIKLAMASI: kitleden rastgele n büyüklüğünde örneklemeler çekilmiştir. "size=n" örneklem büyüklüğünü, "replace=False" seçilen bir gözlemin tekrardan seçilmesini engeller.

YORUM: örneklem büyüklüğü arttıkça elde edilen istatistiklerin parametreye yakınsadığını görmekteyiz.

```
np.random.seed(1)
orneklem_ort1= []

for i in range(1000):

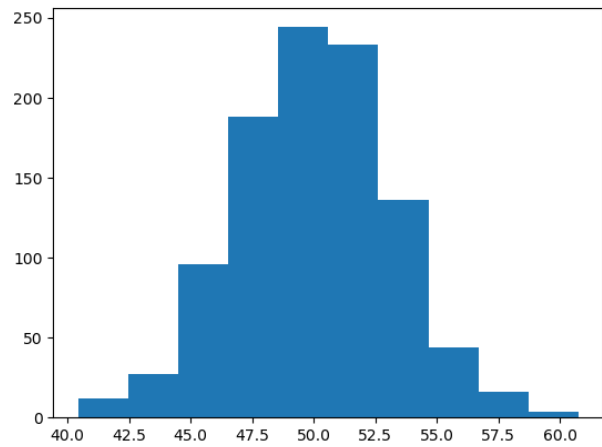
    orneklem = np.random.choice(kitle, size=10, replace=False)
    ortalama = np.mean(orneklem)
    orneklem_ort1.append(ortalama)

rastgele_orneklem_ort1=np.mean(orneklem_ort1)
print(rastgele_orneklem_ort1)

rastgele_orneklem_var1=np.var(orneklem_ort1)
print(rastgele_orneklem_var1)

plt.hist(orneklem_ort1)
plt.show()
```

```
49.98256424906463
10.032684119637677
```



KODUN AÇIKLMASI: örneklem büyüklüğü 10 olan 1000 tane örneklem seçiyoruz. 1000 tane örneklemi for döngüsünde elde ettikten sonra 1000 tane örneklem ortalaması ve varyansını np.ort , np.var kodlarıyla buluyoruz. Matplotlib kütüphanesini kullanarak plt.hist koduyla histogram grafiğini elde ediyoruz.

```

np.random.seed(1)
orneklem_ort2= []

for i in range(1000):

    orneklem = np.random.choice(kitle, size=30, replace=False)
    ortalama = np.mean(orneklem)
    orneklem_ort2.append(ortalama)

rastgele_orneklem_ort2=np.mean(orneklem_ort1)
print(rastgele_orneklem_ort2)
rastgele_orneklem_var2=np.var(orneklem_ort2)
print(rastgele_orneklem_var2)

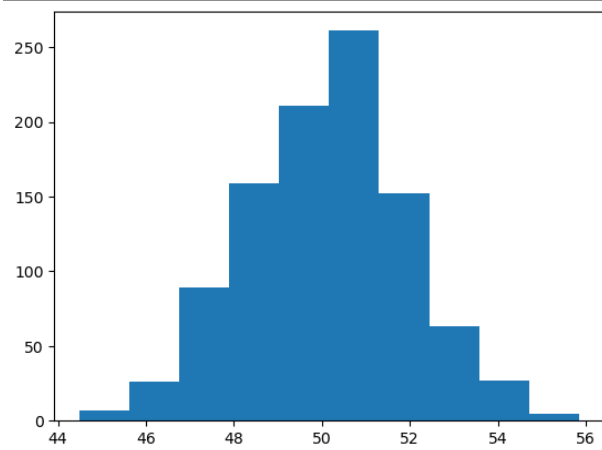
plt.hist(orneklem_ort2)
plt.show()

```

```

49.98256424906463
3.3185730559129696

```



```

np.random.seed(1)
orneklem_ort3= []

for i in range(1000):

    orneklem = np.random.choice(kitle, size=100, replace=False)
    ortalama = np.mean(orneklem)
    orneklem_ort3.append(ortalama)

rastgele_orneklem_ort3=np.mean(orneklem_ort3)
print(rastgele_orneklem_ort3)
rastgele_orneklem_var3=np.var(orneklem_ort3)
print(rastgele_orneklem_var3)

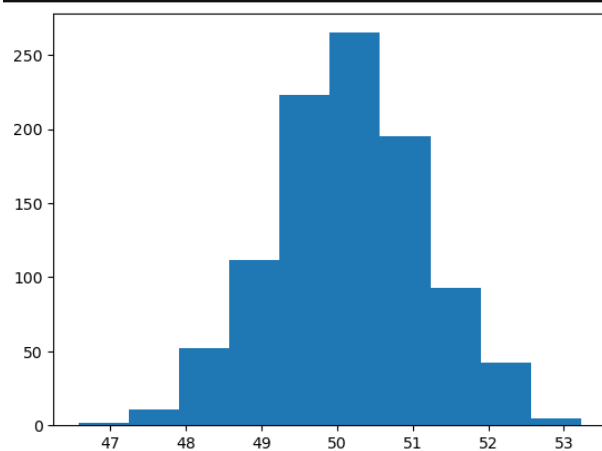
plt.hist(orneklem_ort3)
plt.show()

```

```

50.143893448860055
1.0042986031434284

```



```
np.random.seed(1)
orneklem_ort4= []

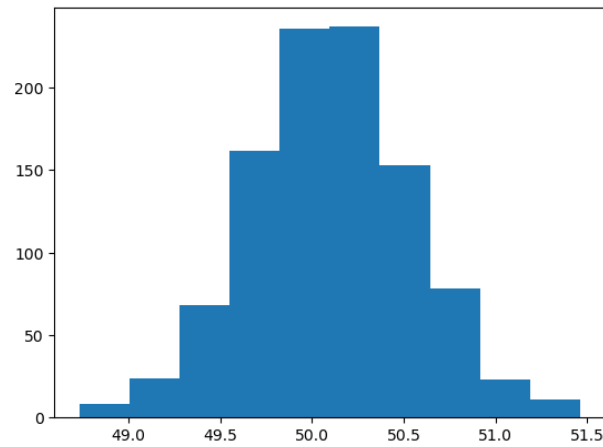
for i in range(1000):

    orneklem = np.random.choice(kitle, size=500, replace=False)
    ortalama = np.mean(orneklem)
    orneklem_ort4.append(ortalama)

rastgele_orneklem_ort4=np.mean(orneklem_ort4)
print(rastgele_orneklem_ort4)
rastgele_orneklem_var4=np.var(orneklem_ort4)
print(rastgele_orneklem_var4)

plt.hist(orneklem_ort4)
plt.show()
```

```
50.10071430946264
0.1972173708885205
```



YORUM: yaptığımız deneyin amacı örneklem büyüklüğü arttıkça elde ettiğimiz istatistiklerin aldığı değerleri gözlemlemektir. Sonuç olarak net bir şekilde gördüğümüz üzere örneklem büyüklüğü arttıkça varyans çok düşük çıkmaktadır. Bu yüzden örneklem büyüklüğü arttıkça tahminlerin güvenilirliğinin arttığını Merkezi limit teoremine göre söyleyebiliriz. Histogram grafiğinde gördüğümüz üzere n=10 iken tabloda değerlerinde 40.0' a kadar değerler alabilirken n=500 olduğunda sadece 50.0'ın yakınında (48.5-51.5) arasında dağıldığını da görmekteyiz.

•Örneklem ortalaması yansız mı yoksa yansız mı?

Cevap: örneklem ortalaması yansızdır çünkü örneklem ortalamasının beklenen değeri parametreye yani kitle ortalamasına eşittir. Simülasyon sonuçlarını da göz önünde bulundurursak kitle ortalamasına çok yakın değerler elde edilmiştir.

• Örneklem ortalaması tutarlı mı?

Cevap: örneklem büyüklüğü sonsuza gider durumunda tahmin edicinin beklenen değeri parametreye eşitse ve tahmin edicinin varyansı 0 ise tahmin edici parametre için tutarlı bir tahmin edicidir. Simülasyon sonuçlarına bakarsak tahmin edicinin beklenen değeri kitle ortalamasına çok yakın değerler almaktadır. Örneklem büyüklükleri arttıkça varyans değerinin de 0'a yakınsadığını açık bir şekilde görmekteyiz. Bu yüzden örneklem ortalaması tutarlıdır.

- Örneklem büyüklüğü arttıkça tahminler parametre etrafında yoğunlaşıyor mu?

Cevap: evet. Örneklem büyüklüğü arttıkça varyans değeri 0'a yaklaştığı için tahminler parametreden çok az sapmaktadır. Bu yüzden parametre etrafında yoğunlaşır diyebiliriz.

- Örneklem büyüklüğü arttıkça tahminlerin varyansı nasıl değişmektedir?

Cevap: simülasyon sonuçlarında net bir şekilde görüldüğü üzere 0'a yaklaşmaktadır.

ÖDEV 2

```
import numpy as np
import pandas as pd

np.random.seed(2)

ort = 20
sigma = 15
varyans = 225
n = 30
R = 1000

ort_T1_list = []
ort_T2_list = []
ort_T3_list = []

var_TV1_list = []
var_TV2_list = []
```

KODUN AÇIKLAMASI: numpy ve pandas kütüphanelerini içe aktarıp, np.random.seed() her çalıştırmada farklı sonuçlar almamak için kullanılan kod.

Soruda verilen ortalama, varyans, standart sapma, örneklem büyüklüğü ve deney sayısını tanımlıyoruz.

Tahmin edicilerden elde edilen değerleri saklamak için boş listeler oluşturduk. Her simülasyon sonrasında listeler o değerleri tutacaktır.

```
for i in range(R):

    orneklem = np.random.normal(ort, sigma, n)

    T1 = np.mean(orneklem)
    T2 = (orneklem[0] + orneklem[1]) / 2
    T3 = (orneklem[0] - 2*orneklem[1] + 3*orneklem[2]) / 6

    TV1 = np.var(orneklem, ddof=1)
    TV2 = ((orneklem[0] - orneklem[1])**2) / 2

    ort_T1_list.append(T1)
    ort_T2_list.append(T2)
    ort_T3_list.append(T3)

    var_TV1_list.append(TV1)
    var_TV2_list.append(TV2)
```

KODUN AÇIKLAMASI:

Burada normal dağılıma göre örneklem çekmek için np.random.normal() kodunu kullandım ve tahmin edicilerin tanımına göre onların değerlerini hesapladım. “ddof=1” kodunu (n-1) serbestlik derecesine bölmek için kullandım. Ardından ürettiğimiz tüm verileri listelerin içine aktardım.

```
T1_ort = np.mean(ort_T1_list)
T2_ort = np.mean(ort_T2_list)
T3_ort = np.mean(ort_T3_list)

T1_yanlilik = T1_ort - ort
T2_yanlilik = T2_ort - ort
T3_yanlilik = T3_ort - ort

T1_var = np.var(ort_T1_list)
T2_var = np.var(ort_T2_list)
T3_var = np.var(ort_T3_list)

T1_hko = np.mean((np.array(ort_T1_list) - ort)**2)
T2_hko = np.mean((np.array(ort_T2_list) - ort)**2)
T3_hko = np.mean((np.array(ort_T3_list) - ort)**2)
```

```
TV1_ort = np.mean(var_TV1_list)
TV2_ort = np.mean(var_TV2_list)

TV1_yanlilik = TV1_ort - varyans
TV2_yanlilik = TV2_ort - varyans

TV1_var = np.var(var_TV1_list)
TV2_var = np.var(var_TV2_list)

TV1_hko = np.mean((np.array(var_TV1_list) - varyans)**2)
TV2_hko = np.mean((np.array(var_TV2_list) - varyans)**2)
```

KODUN AÇIKLAMASI: Her tahmin edici için elde edilen 1000 tahminin ortalamasını alınmıştır. Ardından tahmin ortalaması ile kitle ortalaması arasındaki fark hesaplanmıştır bu değer 0 ise tahmin edici yansızdır. Tahminlerin ortalamadan ne kadar saptığını gözlemlemek için tahmin edicilerin varyansı hesaplanmıştır. Her tahmin edicinin gerçek parametre değerinden ne kadar saptığını ölçer. Bu ölçüm her tahminin gerçek değerden farkının karesiyle bulunur ve bulunun değerlerin ortalaması bize hata kareler ortalamasını verir. HKO hem yanlılığı hem de varyansı birlikte hesapladığı için tahmin edicinin genel başarısı anlamına da gelir.

```
etkinlik_T2 = T1_var / T2_var
etkinlik_T3 = T1_var / T3_var
etkinlik_TV2 = TV1_var / TV2_var
```

```
-> ETKİNLİK
T2: 0.06620051529743268
T3: 0.0854122905520794
TV2: 0.03719671038345685
```

YORUM: T1'in T2'ye etkinliği, T1'in T3'e etkinliği ve TV1'in TV2'ye etkinliği hesaplanmıştır. Bu etkinlik değerlerine bakıldığında etkinlik T1'in T2'ye ve T3'e göre etkinliği hesaplanmıştır. T1 T2'ye göre daha etkindir. Aynı şekilde T1 T3'e göre daha etkindir. Sonuç olarak ortalama için en uygun tahmin edici T1'dir. Aynı şekilde TV1'in TV2'ye göre daha etkin olduğu söylenebilir. Varyans için en iyi tahmin edici de TV2'dir.


```
pd.set_option("display.float_format", "{:.4f}".format)

ortalama_df = pd.DataFrame({
    "Tahmin Edici": ["T1", "T2", "T3"],
    "Ortalama": [T1_ort, T2_ort, T3_ort],
    "Yanlilik": [T1_yanlilik, T2_yanlilik, T3_yanlilik],
    "Varyans": [T1_var, T2_var, T3_var],
    "HKO": [T1_hko, T2_hko, T3_hko]
})

varyans_df = pd.DataFrame({
    "Tahmin Edici": ["TV1", "TV2"],
    "Ortalama": [TV1_ort, TV2_ort],
    "Yanlilik": [TV1_yanlilik, TV2_yanlilik],
    "Varyans": [TV1_var, TV2_var],
    "HKO": [TV1_hko, TV2_hko]
})

print("\n-> ORTALAMA ( $\mu$ ) SONUÇLARI ")
print(ortalama_df)

print("\n-> VARYANS ( $\sigma^2$ ) SONUÇLARI ")
print(varyans_df)

print("\n-> ETKİNLİK")
print("T2:", etkinlik_T2)
print("T3:", etkinlik_T3)
print("TV2:", etkinlik_TV2)
```

KODUN AÇIKLAMASI: pandas kütüphanesinde `pd.set_option("display.float_format", "{:.4f}".format)` kodunu kullandım çünkü ondalık sayılardan sonra çok uzun olduğu için sadece 4 hane gözükmesini ve daha düzenli durması için kullandım. Tablo şeklinde görebilmek içinde `pd.DataFrame` kodunu kullanarak değerleri tablo şeklinde çıktısını almak için kullandım.

```
-> ORTALAMA ( $\mu$ ) SONUÇLARI
Tahmin Edici  Ortalama  Yanlilik  Varyans  HKO
0            T1    19.9489    -0.0511    7.5621    7.5647
1            T2    20.0620    0.0620   114.2299   114.2338
2            T3     6.4932   -13.5068    88.5362   270.9686

-> VARYANS ( $\sigma^2$ ) SONUÇLARI
Tahmin Edici  Ortalama  Yanlilik  Varyans  HKO
0            TV1    226.7897     1.7897   3703.9247   3707.1278
1            TV2    229.8409     4.8409  99576.6731  99600.1076
```

Her iki parametre için hangi tahmin edici daha iyi performans göstermektedir?
Nedenlerini detaylı olarak açıklayınız.

CEVAP: kitle ortalaması için en iyi tahmin edici (T1) örneklem ortalaması'dır. Bunun nedeni örneklem ortalamasının tüm örneklem verilerine sahip olmasıdır. Bu sayede tahminler daha tutarlı ve ortalamaya yakın değerler almaktadır (varyansı küçüktür).

Kitle varyansı için en iyi tahmin edici (TV1) örneklem varyansıdır. Aynı şekilde TV1 tüm örneklem verilerini kullanırken, TV2 sadece 2 veriye dayanmaktadır. Bu sebeple TV2'deki değerlerin daha çok saptığı (varyansı büyük) gözlemlenir. Bu yüzden (TV1) örneklem varyansı tahmin edici olarak tercih edilir.

- Daha düşük Hata Kareler Ortalaması neden daha iyi bir tahmin edici anlamına gelir?

CEVAP: Hata kareler ortalaması, tahmin edicini ortalamadan ne kadar saptığını ölçer. Hata kareler ortalaması ne kadar düşükse ortalamaya daha yakın değerler alır ve sapma o kadar az olur. Sonuç olarak hata kareler ortalaması düşük olan daha iyi bir tahmin edici olur.

ÖDEV 3

```
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(1)

teta_tahminleri = []

for i in range(10000):
    orneklem = np.random.uniform(low=0, high=10, size=30)
    teta_tahmin = np.max(orneklem)
    teta_tahminleri.append(teta_tahmin)

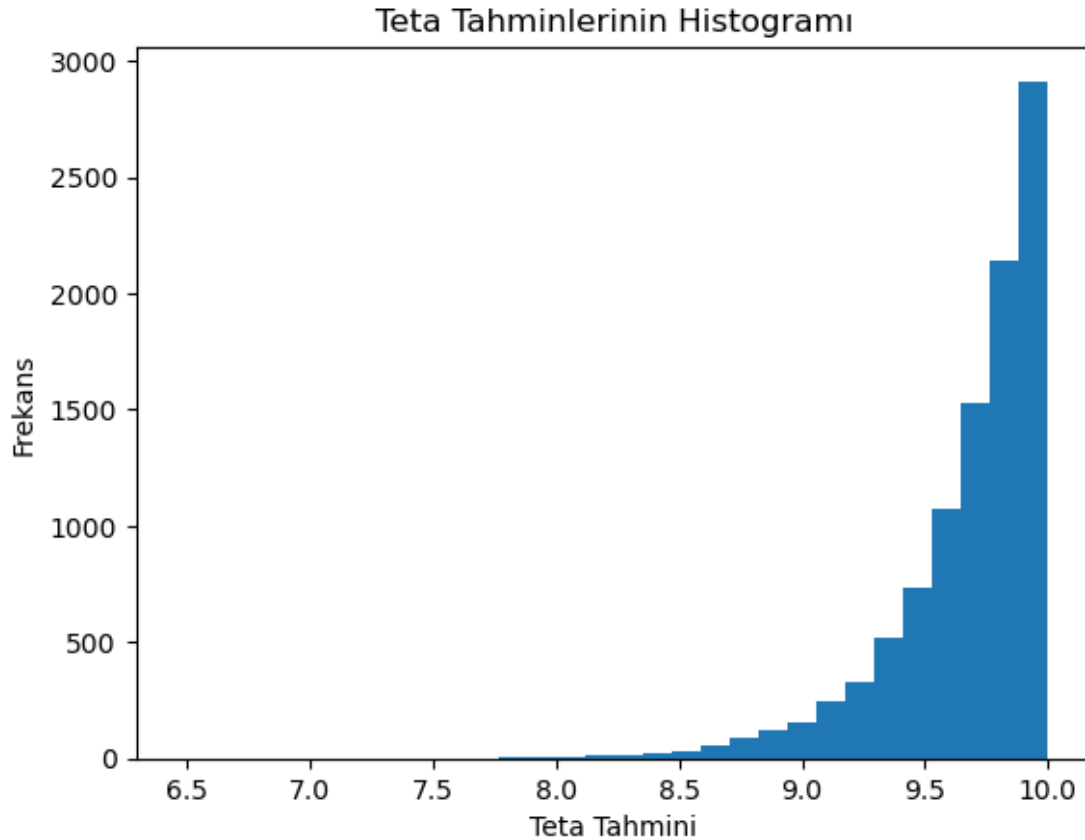
ortalama = np.mean(teta_tahminleri)
print("Teta tahminlerinin ortalaması:", ortalama)

varyans = np.var(teta_tahminleri)
print("Teta tahminlerinin varyansı:", varyans)

plt.hist(teta_tahminleri, bins=30)
plt.title("Teta Tahminlerinin Histogramı")
plt.xlabel("Teta Tahmini")
plt.ylabel("Frekans")
plt.show()
```

KODUN AÇIKLAMASI: uniform dağılımdan minimum 0 değerini, maksimum 10 değerini alan 30 gözlem alınmıştır bu da np.random.uniform() koduyla yapılmaktadır. Bu deney 10000 defa yapılmıştır. Teta tahmin edicisi 30 gözlem arasından maksimum değeri alması için np.max kodu yazılmıştır. Teta tahmin edicisinin ortalaması np.mean , varyansını np.var ile buluruz. Matplotlib kütüphanesi kullanılarak plt.hist koduyla histogram grafiği çizilmiştir.

```
Teta tahminlerinin ortalaması: 9.676078131873952
Teta tahminlerinin varyansı: 0.09776105975079627
```



YORUM: Teta tahmin edicisinin aldığı değerlerin ortalaması alabileceği maksimum değere (10) çok yakın değerler almıştır ama tahmin edici hiçbir zaman parametreden büyük değerler alamaz. Bu nedenle tahminler 10'a yakın değerler almıştır ve sola çarpık bir grafik ortaya çıkmıştır.

- En büyük sıralı istatistik, neden θ için iyi bir tahmin edicidir?

CEVAP: En büyük sıralı istatistik ve parametre arasındaki fark ne kadar küçükse kullanılan tahmin edici o kadar iyi bir tahmin edicidir. Örnekte gördüğümüz üzere en büyük sıralı istatistikle parametre arasındaki fark çok azdır.

- Bu tahmin edicinin yanlı olup olmadığını araştırınız.

CEVAP: Tahmin edicinin yansız olması için beklenen değer parametreye eşit olması gerekir. Bu tahmin edici hiçbir zaman parametreye eşit olamaz. Bu yüzden tahmin edici yanlıdır.

- Örneklem büyüklüğü arttıkça tahmin davranışı nasıl değişmektedir? Bu durum hangi özellik ile açıklanır?

CEVAP: Tutarlılık özelliği ile açıklanır. Tutarlılık teoremi örneklem büyüklüğü arttıkça (limit n sonsuza giderken) tahmin edicinin beklenen değeri parametreye yaklaşıyor ve tahmin edicinin varyansı 0'a yaklaşıyor. Bu yüzden örneklem büyüklüğü arttıkça tahmin edici parametre gibi davranır.